C++ ile Diziler ve Dinamik Bellek

Dinamik Bellek

Biligisayar Programlama

Fizik Bölümü

22 Ekim 2026

Fizikçiler Neden Bellek Yönetimini Bilmeli?

Fiziksel problemleri sayısal yöntemlerle çözerken veriyi bellekte nasıl tuttuğumuz, simülasyonun çalışma hızını ve çözünürlüğünü doğrudan belirler:

N-Cisim Problemi

Milyonlarca yıldız veya parçacık için konum (x, y, z) ve momentum bileşenlerinin ardışık saklanması.

Ayrık Uzay Ağları

Dalga ve ısı diferansiyel denklemlerinin çözümü için oluşturulan 2D/3D uzay-zaman ızgaraları.

Deney Verisi Akışı

Parçacık hızlandırıcı veya dedektörlerden zamanla artan, boyutu önceden bilinmeyen ölçüm serileri.

Statik Diziler: Kesintisiz Bellek Blokları

Statik diziler, aynı türden verilerin bellekte art arda (contiguous) yerleştiği en temel veri yapılarıdır:

```
// 3 boyutlu parçacık pozisyon vektörü (r)
double r[3] = {1.2, 0.0, -4.5};
```

Bellek Yerleşimi (Adres & Değer):

r[0] (Adres: 0x1000)	**r[1]** (Adres: 0x1008)	**r[2]** (Adres: 0x1010)
1.2 (x bileşeni) 8 bayt	0.0 (y bileşeni) 8 bayt	-4.5 (z bileşeni) 8 bayt

- **Doğrudan Erişim:** İndis adresi Adres = Başlangıç + ($i \times$ Eleman_Boyutu) formülüyle $\mathcal{O}(1)$ zamanda hesaplanır.

Statik Dizilerin Kırılma Noktası

Statik diziler hızlıdır ancak büyük bilimsel hesaplamalarda iki temel sınıra takılırlar:

Yığın (Stack) Sınırı: ∼1–8 MB

Statik yerel değişkenler Stack bölgesinde tutulur. Büyük fiziksel alanlar için ayrılan bellek stack alanını hemen aşar:

```
double potansiyel[2000][2000];
```

$\rightarrow 4 \times 10^6 \times 8$ bayt ≈ 32 MB

Sonuç: Stack Overflow (Çökme)

Esneklik Eksikliği

- Parçacık sayısı N derleme anında sabit olmak zorundadır.
- Kullanıcıdan veya veri dosyasından gelen dinamik parçacık sayısına uyum sağlayamaz.
- Gereğinden büyük tanımlamak belleği israf eder, küçük tanımlamak simülasyonu sınırlar.

Bellek Ayrımı: Stack (Yığın) vs Heap (Öbek)

Bilgisayar belleği fiziksel simülasyonlar açısından iki ana bölgeye ayrılır:

Stack (Yığın)

- **Yönetim:** Otomatik tahsis ve temizleme.
- **Erişim Hızı:** Çok hızlı (CPU register'a yakın hız).
- **Kapasite:** Oldukça küçük (genelde 1–8 MB).
- **Kullanım:** Küçük vektörler (3×3 matris, x, y, z).

Heap (Öbek)

- **Yönetim:** Yazılımcı sorumluluğunda (`new` / `delete`).
- **Erişim Hızı:** İşaretçi (pointer) dolayımıyla erişim.
- **Kapasite:** Sistemdeki boş RAM boyutu kadar (GB'lar).
- **Kullanım:** Büyük ızgaralar, milyonlarca parçacık.

Dinamik Bellek Yönetimi: new[] ve delete[]

Çalışma anında boyutu belirlenen diziler için Heap üzerinden alan tahsis edilir:

```
int N;
cout << "Simulasyondaki parcacik sayisini giriniz: ";
cin >> N;

// Heap üzerinde N adet double alanı tahsis edilir
double* momentum = new double[N];

momentum[0] = 3.14e-24; // Klasik dizi gibi erisim

// Tahsis edilen alan mutlaka serbest birakilmalidir
delete[] momentum;
momentum = nullptr; // Dangling pointer onlemi
```

- `new[]` Heap üzerinde bitişik bellek bloğu açar ve başlangıç adresini işaretçiye (`double*`) döner.
- `delete[]` açılan alanı işletim sistemine geri iade eder.

Kritik Hata: Bellek Sızıntısı (Memory Leak)

Simülasyon hesaplamalarında yapılan en tehlikeli hata, tahsis edilen alanın serbest bırakılmamasıdır:

Simülasyon Döngüsünde Bellek Sızıntısı

Zaman adımlı bir simülasyon ($t = 0 \rightarrow t_{\text{son}}$) döngüsü içerisinde her adımda `new` yapılıp `delete` unutulursa:

- Her adımda sistem RAM'i hızla dolmaya başlar.
- Bir süre sonra işletim sistemi programı *Out of Memory (OOM Killer)* ile zorla kapatır.
- Saatler süren uzun bir deney veya simülasyon yarıda kalır.

Altın Kural: Her `new[]` çağrısının mutlak karşılığı olan bir `delete[]` çağrısı olmalıdır!

2 Boyutlu Alan ve Isı Dağılımı Modeli

İki boyutlu bir uzay ızgarasını dinamik olarak kurmanın iki yöntemi vardır:

1. İşaretçi Dizisi (`double**`)

```
double** grid = new double*[satir];
for(int i = 0; i < satir; ++i)
grid[i] = new double[sutun];

// Erisim: grid[i][j]
```

Dezavantaj: Bellekte parçalı yerleşir, satır satır temizlik gerektirir.

2. Düzleştirilmiş Dizi (Tavsiye Edilen)

```
double* grid = new double[satir *
    sutun];

// Erisim (i, j):
// index = i * sutun + j
grid[i * sutun + j] = 100.0;
```

Avantaj: Bellekte tamamen kesintisizdir, CPU önbellek (cache) dostudur ve tek bir `delete[]` ile silinir.

Modern Çözüm: :vector

Modern C++'ta manuel `new`/`delete` yerine RAII prensibine dayanan standart kütüphane araçları tercih edilir:

```
#include <vector>

vector<double> enerji(N, 0.0); // N elemanli, 0.0 baslangic degeri
enerji.push_back(1.6e-19);          // Calisma aninda eleman ekleme
```

Otomatik Temizlik

Kapsamdan (scope) çıkınca belleği otomatik serbest bırakır. Bellek sızıntısı riski yoktur.

Esnek Boyutlandırma

Yeni dedektör verisi geldikçe kendini otomatik olarak yeniden boyutlandırabilir.

Yüksek Performans

Verileri Heap üzerinde yine bitişik tuttuğundan saf C dizileri kadar hızlı çalışır.

Hangi Durumda Hangi Yapı Tercih Edilmeli?

Yapı Türü	**Boyut Belirlenimi**	**Bellek Bölgesi**	**Fizikteki Örnek Kullanım**
C Dizisi (`double[]`)	Derleme anı (Sabit)	Stack	3D Vektörler, 4×4 Metrik Tenisör
`new[]` / `delete[]`	Çalışma anı (Dinamik)	Heap	Çok büyük sabit boyutlu uzay ağları
`vector`	Çalışma anı (Büyüyebilir)	Heap (Yönetimli)	Dedektör akışı, parçacık listeleri

"Fizik kodlamasında varsayılan tercihiniz daima `vector` olmalıdır."

Hesaplamalı Fizikte Bellek Disiplini: Altın Kurallar

1. **Döngü Dışında Tahsis Edin:** Zaman adım döngüsü (`while(t < t_max)`) içinde asla `new` çağırmayın. Alanı döngüden önce ayırıp tekrar kullanın.
2. **Önbellek (Cache) Dostu Döngüler:** İki boyutlu matrisleri tararken satır bazlı (row-major: dışta i, içte j) dolaşın. Ters sıralama simülasyonu 10 kata kadar yavaşlatabilir.
3. **Her `new[]` İçin Bir `delete[]`:** Çift silme (double delete) veya serbest bırakmayı unutma (memory leak) hatalarından kaçının.
4. **Modern C++ İle Güvenlik:** Mümkün olan her yerde doğrudan işaretçiler yerine `vector` kullanın.

Sorular & Tartışma

Laboratuvar Uygulaması: 2 Boyutlu Isı Yayılım Denkleminin Dinamik Matris ile Modellenmesi

```
$ g++ -O3 isi_simulasyonu.cpp -o sim
$ ./sim
```